VAASAN AMMATTIKORKEAKOULU
UNIVERSITY OF APPLIED SCIENCES

Vladislav Rumiantsev

BLUETOOTH MESH NETWORK USING RASPBERRY PI

School of Technology
2024

VAASAN AMMATTIKORKEAKOULU
UNIVERSITY OF APPLIED SCIENCES
Information Technology, Bachelor of Engineering

ABSTRACT

Author	Vladislav Rumiantsev
Title	Bluetooth Mesh Network using Raspberry Pi
Year	2024
Language	English
Pages	47
Name of Supervisor	Smail Menani

This thesis explores the implementation of a Bluetooth mesh network using Raspberry Pi device, focusing on the practical deployment and utilization of the mesh networking technology.

The background highlights the increasing demand for reliable and scalable wireless communication solutions, particularly in IoT applications.

The aim of the thesis is to investigate the hardware and software components required for establishing a Bluetooth mesh network, including the provisioning process, node configuration, and data transmission mechanisms.

Material and methods involve the installation of Raspberry Pi OS, recompilation of the kernel to enable support for encryption algorithms, and deployment of the Meshctl tool for mesh network management, as well as developing own Web UI Application for it.

The results demonstrate successful provisioning and configuration of nodes, along with the establishment of communication channels within the mesh network.

Conclusions drawn from the study underscore the feasibility and effectiveness of Bluetooth mesh networking using Raspberry Pi, offering insights into its practical implementation for various IoT applications.

Keywords IoT, wireless communication, practical implementation

CONTENTS

1 INTRODUCTION 8

2 BLUETOOTH MESH 9

2.1 Overview 9

2.2 Bluetooth Mesh Architecture 9

2.2.1 Main Principles 9

2.2.2 Node Roles 10

2.2.3 Provisioning Process 11

2.2.4 Network Keys 12

2.2.5 Elements and Models 13

2.2.6 Binding, Subscribing and Publishing 15

2.3 Bluetooth Mesh Data Transmission 16

2.3.1 Structure 16

2.3.2 Access Layer 16

2.3.3 Upper Transport Layer 18

2.3.4 Lower Transport layer 18

2.3.5 Network Layer 19

2.3.6 Data Encryption 22

2.3.7 Receiving and Transmitting Data 24

2.3.8 GATT 26

2.4 Comparison to Zigbee 27

3 HARDWARE AND SOFTWARE COMPONENTS 29

3.1 Hardware 29

3.1.1 Provisioner 29

3.1.2 Node 30

3.2 Software 31

3.2.1 Operating System 31

3.2.2 Bluetooth Stack on GNU/Linux 32

3.2.3 Meshctl Tool 32

4 PREPARATION 34

4.1 Raspberry Pi 5 34
4.1.1 Installation of Raspberry Pi OS 34
4.1.2 Recompilation of Kernel 34
4.1.3 Installation of BlueZ and Meshctl 34
4.2 Nordic Thingy 53 35
4.2.1 Installation of Programmer 35
4.2.2 Choosing the Correct Application 35
5 PRACTICAL USAGE 36
5.1 Example Scenario 36
5.1.1 Set-up Switch and Light 36
5.2 Implementation in Meshctl 36
5.2.1 Overview 36
5.2.2 Implementation 40
5.3 Implementation In Web Application 43
5.3.1 Overview 43
5.3.2 Implementation 45
6 CONCLUSIONS 47
REFERENCES 48

LIST OF FIGURES AND TABLES

Figure 1. Remote Provisioning 12
Figure 2. Nodes with elements 13
Figure 3. A node with elements and models 14
Figure 4. Structure of Data Transmission 16
Figure 5. Encryption of Transmitted message 22
Figure 6. Encryption of Application Payload 23
Figure 7. Encryption of Transport 23
Figure 8. Layer Obfuscation Technique 24
Figure 9. Filtering stage 25
Figure 10. Raspberry Pi 5 29
Figure 11. Nordic Thingy 53 31
Figure 12. Meshctl tool showing its version 33
Figure 13. Switch and Light example 36
Figure 14. Discovering and provisioning a device by using meshctl 40
Figure 15. Successfully provisioned device in meshctl 41
Figure 16. Set up for light node 42
Figure 17. Set up for Switch node 42
Figure 18. Discovering and provisioning node in Web Application 43
Figure 19. Nodes page in Web Application 44
Figure 20. Keys page in Web Application 45
Figure 21. Status Page in Web Application 45
Figure 22. Set up Light and Switch nodes in Web Application 46

Table 1. Access Layer PDU 17
Table 2. Upper Transport Layer PDU 18
Table 3. PDU of Unsegmented Message for Lower Network Layer 19
Table 4. Network Layer PDU 19
Table 5. Example of the Smallest Network Layer Packet 21
Table 6. Comparison of Bluetooth Mesh and Zigbee 28
Table 7. Meshctl Main Menu 37
Table 8. Meshctl Config Menu 38
Table 9. Meshctl Menu onoff 40

CONCEPTS AND ABBREVIATIONS

GNU/Linux	A family of operating systems that are based on Linux kernel
Distribution	A premade OS that is based on GNU/Linux
Debian	A GNU/Linux Distribution
Raspberry Pi OS	An operating system that is based on Debian Distribution
Backend	A data access layer in a piece of software
Frontend	A presentation layer in a piece of software
Python	A High-level programming language
Javascript	A programming language for Web
React	A library for Javascript for building User Interfaces
Next.js	A framework for React

OS	Operating System
IoT	Internet Of Things
SoC	System-On-Chip
PDU	Protocol Data Unit
BLE	Bluetooth Low Energy
HVAC	Heating, ventilation, and air conditioning
UI	User Interface
CPU	Central Processing Unit
ARM	Advanced RISC Machines
IEEE	Institute of Electrical and Electronics Engineers
CLI	Command-Line Interface
LED	Light-Emitting Diode

1 INTRODUCTION

This thesis explores the implementation of a Bluetooth Mesh network on Raspberry Pi 5, accompanied by the development of graphical web software for network management. Through experimentation and analysis, it assesses the feasibility and effectiveness of deploying Bluetooth Mesh technology on Raspberry Pi hardware. The project aims to provide insights into the practical integration of Bluetooth Mesh networks in IoT applications, leveraging the versatility and accessibility of Raspberry Pi platforms for network management and control.

The thesis expects to demonstrate the successful implementation of Bluetooth Mesh networking on Raspberry Pi 5, showcasing reliable network performance and scalability. Additionally, the developed graphical web software is anticipated to provide intuitive management capabilities for configuring and monitoring the Bluetooth Mesh network. The project aims to offer valuable insights into the potential applications and benefits of deploying Bluetooth Mesh technology on Raspberry Pi platform in various IoT environments.

2 BLUETOOTH MESH

2.1 Overview

Bluetooth Mesh is a robust wireless communication protocol designed for creating large-scale networks of smart devices. Unlike traditional point-to-point Bluetooth connections, Bluetooth Mesh enables devices to communicate with each other in a mesh topology, allowing for seamless and reliable communication across a wide area. This technology is particularly well-suited for applications in the IoT and smart home automation, where numerous devices need to interact and coordinate with each other. By forming self-healing and self-optimizing networks, Bluetooth Mesh facilitates efficient data exchange and enables advanced functionalities, such as group control, remote monitoring, and automation.

It uses the BLE protocol, operating in the 2.4 GHz frequency band, to facilitate communication between devices. Bluetooth Mesh employs a managed flooding technique, where messages are relayed by neighbouring nodes to ensure widespread coverage and redundancy. Each device in the mesh network is assigned a unique 128-bit identifier, allowing for secure and reliable communication [1]. Furthermore, Bluetooth Mesh supports various features such as message segmentation, retransmissions, and acknowledgments to ensure robust data transmission even in challenging environments. These technical aspects contribute to the scalability, reliability, and interoperability of Bluetooth Mesh networks in diverse IoT applications.

2.2 Bluetooth Mesh Architecture

2.2.1 Main Principles

Bluetooth Mesh architecture delineates the structural framework and coordination of devices within a mesh network, a dynamic topology connecting nodes directly and non-hierarchically. This network design establishes multiple intercon-

nected routes between nodes, fostering robustness and reliability by ensuring redundant pathways for data transmission. Employing a flooding technique, data originating from a single node permeates throughout the entire network, enhancing information dissemination and resilience. This architectural approach underpins the scalability and resilience of Bluetooth Mesh networks, facilitating efficient communication and coordination among interconnected devices.

2.2.2 Node Roles

A Node is a device that participates in the mesh network. Each node typically consists of one or more elements, which can represent individual devices or groups of devices [6]. Nodes communicate with each other using the Bluetooth Mesh protocol to relay messages, control devices, and exchange data. Nodes can take on various roles within the network, such as publishers, subscribers, or relays, contributing to the overall functionality and efficiency of the mesh ecosystem.

In Bluetooth Mesh networks, various nodes undertake distinct roles, each contributing to the network's functionality and efficiency.

Relay Nodes serve as intermediaries for forwarding messages between devices within the network. These nodes facilitate communication by relaying data packets, thereby extending the reach of the network and enhancing its coverage area.

Proxy Nodes bridge the gap between Bluetooth Mesh networks and external devices or services. They facilitate communication between Bluetooth Mesh devices and devices that do not support the Bluetooth Mesh protocol, enabling interoperability with legacy Bluetooth devices and internet-based services.

Low Power Nodes (LPNs) are energy-efficient devices designed to operate with minimal power consumption. These nodes periodically wake up to transmit or receive data, conserving energy during idle periods to prolong battery life. LPNs are ideal for battery-powered devices in IoT applications where power efficiency is paramount.

Friend Nodes provide additional support for LPNs by buffering and relaying messages on their behalf, allowing LPNs to remain in a low-power sleep state for extended periods. This role enhances the efficiency of LPNs by minimizing the need for frequent wake-ups, thereby conserving energy and extending battery life in resource-constrained devices.

2.2.3 Provisioning Process

The process of integrating a device into a Bluetooth Mesh network, known as provisioning, is facilitated by a specialized entity called a Provisioner. This procedure involves the transfer of crucial data, such as the network's primary network key (NetKey) and a unique range of unicast addresses, from the Provisioner to the device undergoing provisioning, termed the Provisionee.

Provisioning Protocol, dedicated solely to this task, encompasses various stages, including beaconing, invitation, exchange of public keys, authentication, and distribution of provisioning data. These stages ensure a secure and seamless integration of new devices into the network. [7.] [8.]

Additionally, Bluetooth Mesh 1.1 introduced Remote Provisioning (RPR), enabling provisioning and re-provisioning of nodes not in direct radio range of the Provisioner. [12.] RPR offers multi-hop device provisioning, change of ownership procedures, and dynamic device composition capabilities, enhancing the flexibility and scalability of Bluetooth Mesh networks (Figure 1). [2.]

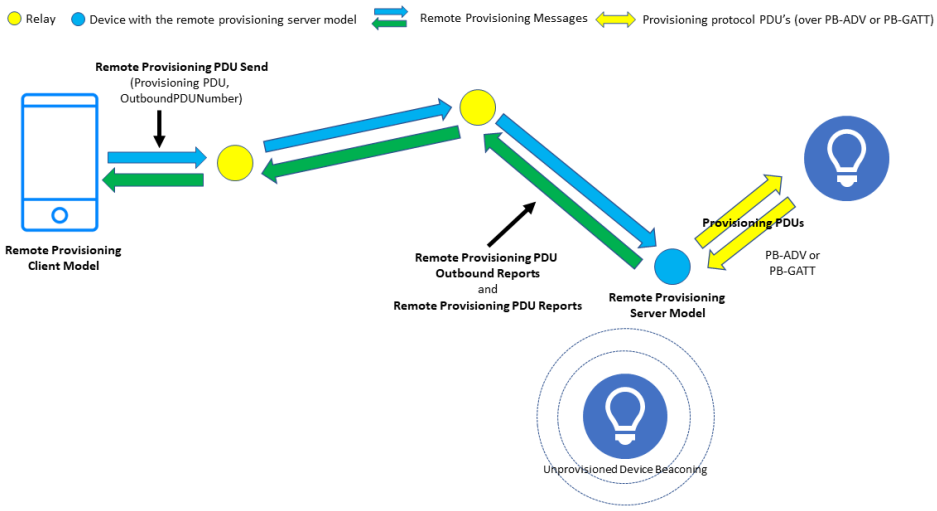

Figure 1. Remote Provisioning

The technical architecture of remote provisioning involves new models like the Remote Provisioning Client and Server, along with the Node Provisioning Protocol Interface, enabling procedures such as Device Key Refresh, Node Address Refresh, and Node Composition Refresh. These procedures allow for key management, address reassignment, and dynamic adaptation to changes in device composition, ensuring the network's security and adaptability in real-world scenarios.

2.2.4 Network Keys

In the realm of Bluetooth Mesh networks, network keys play a pivotal role in ensuring secure communication and data integrity. These keys are utilized for encrypting and authenticating network layer data, signifying a node's membership within a specific subnet.

Each network key is associated with a particular level of access and privileges within the network. Possession of a network key enables a node to encrypt and decrypt messages destined for the network layer, safeguarding against unauthorized access and tampering.

Moreover, application keys, a subset of network keys, are tailored for higher layers of the Bluetooth Mesh protocol stack. These keys are specifically assigned to devices participating in distinct building systems, such as lighting or HVAC systems, enabling fine-grained control and management.

The Bluetooth Mesh protocol allows nodes to possess multiple network keys, each serving unique purposes and catering to different network segments or applications. This multi-key architecture enhances security and segmentation capabilities, allowing for robust and scalable network deployments.

Furthermore, the introduction of the Key Refresh procedure in Bluetooth Mesh 1.0 facilitates the periodic rotation of network keys across the entire network, bolstering security measures and mitigating potential vulnerabilities. However, it is noteworthy that while network keys can be refreshed, there's no provision for changing device keys in Bluetooth Mesh 1.0. [12.]

2.2.5 Elements and Models

In the architecture of Bluetooth Mesh networks, nodes are structured hierarchically into elements and models, forming the fundamental building blocks of device functionality and communication [13]. Each node in this structure has at least one element that is called “Primary Element” as shown in figure 2.

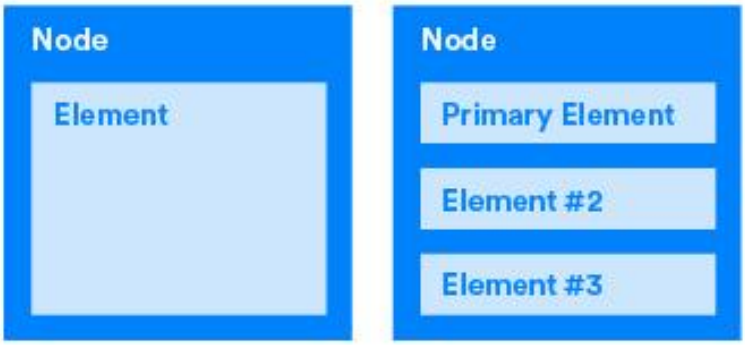

Figure 2. Nodes with elements

An element represents an individually addressable component within a node, capable of performing specific tasks or functionalities. Each element possesses a unique unicast address, serving as the source for all messages originating from that element. Elements are essential for organizing and managing device functionalities in a granular manner, enabling precise control and monitoring.

Models, on the other hand, are standardized software components that define the behaviour and capabilities of an element (figure 3). These models encapsulate specific functionalities, such as switching on/off a device, adjusting brightness, or gathering sensor data. Each model is associated with a set of states and messages, delineating its input-output interface and interaction protocols. [4.]

Figure 3. A node with elements and models

Nodes may consist of multiple elements, each hosting one or more models to fulfil diverse functionalities. The relationship between nodes, elements, and models follows a hierarchical structure, facilitating modular design and extensibility. Typically, nodes contain multiple elements, and each element houses one or more models, creating a flexible and scalable architecture.

The composition of nodes, including the number of elements and the models they contain, can evolve over time to accommodate changes in device configurations or network requirements. This dynamic configurability enables seamless integration of new components, such as sensors or actuators, without necessitating node re-provisioning or reconfiguration.

2.2.6 Binding, Subscribing and Publishing

In Bluetooth mesh networking, binding, subscribing, and publishing are fundamental concepts that govern how devices communicate within the mesh network.

Binding involves associating an application key with a specific model on a node. An application key is a cryptographic key used for encrypting and decrypting application-level data within the mesh network. By binding an application key to a model, the node establishes a secure communication link between the model and the key, ensuring that only authorized devices can interact with the model's functionality.

Subscribing refers to the process by which a node expresses interest in receiving messages published by other nodes within the mesh network. When a node subscribes to a particular model, it effectively requests to receive messages related to that model's functionality. Subscribing allows nodes to stay updated on relevant information and events occurring within the mesh network, enabling them to respond appropriately to changing conditions.

Publishing involves the transmission of messages by a node to inform other nodes within the mesh network about specific events or data. When a node publishes a message, it selects a model and associated message type to convey relevant information to other nodes. Published messages can include status updates, sensor data, control commands, or any other information pertinent to the application running on the node. By publishing messages, nodes contribute to the collective

intelligence of the mesh network, facilitating coordinated operation and distributed decision-making.

2.3 Bluetooth Mesh Data Transmission

2.3.1 Structure

In Bluetooth mesh networking, data transmission follows a hierarchical structure as it traverses through different layers of the protocol stack as shown in figure 4. [12.]

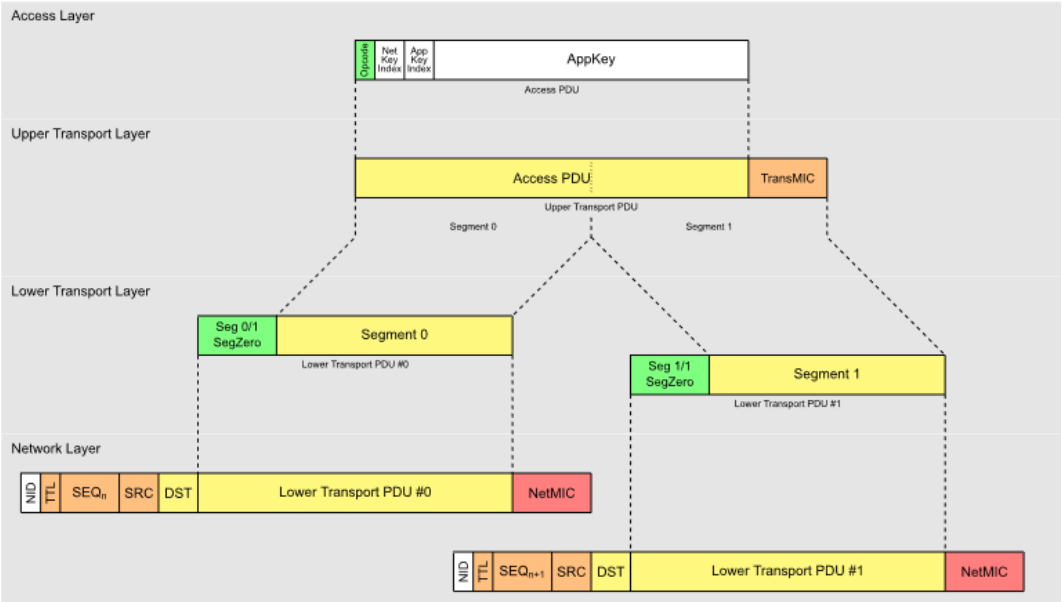

Figure 4. Structure of Data Transmission

2.3.2 Access Layer

The access layer in Bluetooth Mesh is responsible for managing the transmission and reception of messages between nodes within the network.

Table 1. Access Layer PDU

Field Name	Octets
Opcode	1-3
Parameters	0-379

Opcode specifies the operation to be performed and is represented by 1, 2, or 3 octets. The format of the opcode is determined by the most significant bits of the first octet:

- If the most significant bit is zero, it is a 1-octet opcode.
- If the two most significant bits are 10, it is a 2-octet opcode.
- If the two most significant bits are 11, it is a 3-octet opcode.

The opcode allocation specifies how opcodes are structured and allocated within the Bluetooth Mesh network. One-octet opcodes are reserved for Bluetooth SIG defined application opcodes, providing 127 available codes. Similarly, two-octet opcodes are also used for Bluetooth SIG defined application opcodes, offering 16384 available codes. Additionally, three-octet opcodes are designated for manufacturer-specific opcodes, allowing 64 codes per company identifier. The company identifiers, 16-bit values defined by the Bluetooth SIG, are coded into the second and third octets of the three-octet opcodes, using endianness as defined in the specification.

The parameters field contains application-specific data associated with the opcode. It can vary in size from 0 to 379 octets, depending on the opcode size and the maximum allowable payload size.

2.3.3 Upper Transport Layer

The upper layer in the Bluetooth Mesh network architecture encompasses various functionalities such as friend management, low-power messaging, and segmentation and reassembly. At this layer, communication between nodes involves the exchange of messages that contain specific data payloads.

Table 2. Upper Transport Layer PDU

Field Name	Octets
Encrypted Access Payload	1-380
TransMIC	4/8

The Protocol Data Unit at the upper layer consists of the Access Payload, which is composed of an Opcode and optional parameters. It has an encryption that prevents unauthorized access to the payload contents.

In addition to it, the TransMIC (Transmitted Message Integrity Code) provides a cryptographic checksum to verify the integrity of the transmitted message. The TransMIC is appended to the Encrypted Access Payload and is used by the receiving node to detect any tampering or corruption of the message during transmission.

2.3.4 Lower Transport layer

The lower transport layer is pivotal in managing the segmentation and reassembly of Upper Transport PDUs within the Bluetooth mesh network. It features segmented and unsegmented types of messages.

Segmented messages are utilized when the Upper Transport PDU exceeds the maximum size for a single Lower Transport PDU, dividing it into manageable parts.

In contrast, unsegmented messages are employed when the Upper Transport PDU fits within a single Lower Transport PDU.

Table 3. PDU of Unsegmented Message for Lower Network Layer

Field Name	Bits
SEG	1
AKF	1
AID	6
Upper Transport Access PDU	40-120

2.3.5 Network Layer

The network layer serves as the fundamental unit of data transmission within the Bluetooth Mesh network. It encapsulates higher-layer data and metadata necessary for routing and security.

Table 4. Network Layer PDU

Field Name	Bits
IVI (Initialization Vector Index)	1
NID (Network Identifier):	7
CTL (Control)	1
TTL (Time To Live)	7

SEQ (Sequence Number)	24
SRC (Source Address)	16
DST (Destination Address)	16
TransportPDU	8-128
NetMIC (Network Message Integrity Check)	32/64

The **Initialization Vector Index (IVI)**, represented by a single bit, plays a crucial role in encryption by indicating the least significant bit of the Initialization Vector (IV) Index. This index is utilized during encryption processes to ensure the uniqueness and integrity of encrypted data.

Next, the **Network Identifier (NID)**, a 7-bit value derived from the Network Key, serves as a key identifier facilitating the selection of appropriate encryption and privacy keys for securing the PDU. By associating each PDU with a specific NID, the network can effectively manage encryption and decryption processes.

The **Control (CTL)** flag, a 1-bit field, distinguishes between Control Messages and Access Messages within the network. This differentiation influences the size and complexity of the Message Integrity Check (NetMIC), contributing to message authentication and data integrity.

The **Time To Live (TTL)** field, comprising 7 bits, dictates the permissible number of hops a PDU can traverse within the network. By limiting the TTL value, the network can control the propagation of messages, preventing infinite loops and optimizing network efficiency.

The **Sequence Number (SEQ)**, represented by a 24-bit integer, ensures the uniqueness of each originating node's PDUs. By assigning a distinct sequence number to each PDU, the network can accurately track message delivery and detect duplicate or out-of-order packets.

The **Source Address (SRC)**, a 16-bit identifier, denotes the originator of the Network PDU. This address enables receivers to identify the sender of a message, facilitating communication and routing processes within the network.

Conversely, the **Destination Address (DST)**, also a 16-bit identifier, specifies the intended recipient(s) of the PDU. Whether it is a unicast address, group address, or virtual address, the DST field ensures that messages reach their intended targets efficiently.

The **TransportPDU** field contains the actual payload of the PDU, accommodating variable lengths of data. While limited to 128 bits for Access Messages and 96 bits for Control Messages, this field carries essential information and commands necessary for network operation.

Lastly, the **Network Message Integrity Check (NetMIC)**, comprising either 32 or 64 bits, ensures the integrity and authenticity of the Destination Address and TransportPDU fields. By validating the integrity of transmitted data, NetMIC safeguards against tampering and unauthorized access.

Table 5. Example of the Smallest Network Layer Packet

<table>
<tr><td colspan="2">8</td><td colspan="2">16</td><td colspan="3">24</td><td>32</td><td>40</td><td>48</td><td>56</td></tr>
<tr><td>IVI</td><td>NID</td><td>CTL</td><td>TTL</td><td colspan="5">SEQ</td><td colspan="2">SRC</td></tr>
<tr><td colspan="4">DST</td><td colspan="3">TransportPDU</td><td colspan="4">NetMIC</td></tr>
</table>

2.3.6 Data Encryption

Bluetooth Mesh employs AES-CCM encryption at both the network layer and upper transport layer to ensure message confidentiality and integrity. The process is depicted in figure 5.

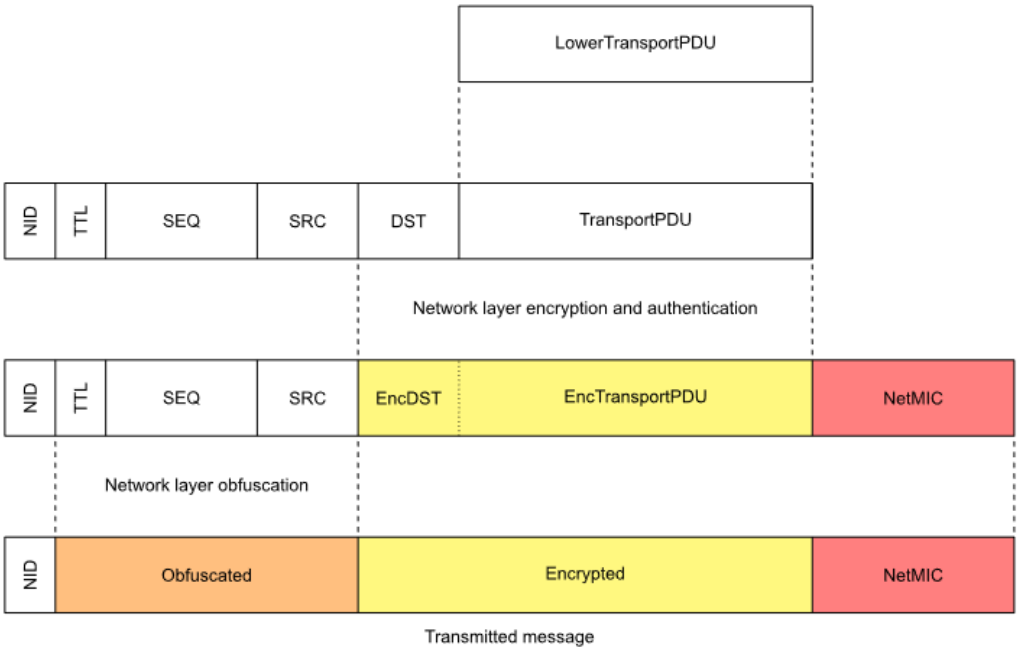

Figure 5. Encryption of Transmitted message

At the upper transport layer, the access payload undergoes encryption and authentication using AES-CCM, similar to BLE encryption methods. Depending on whether the access payload is secured using the application key or device key, the payload is encrypted using the corresponding nonce and key. This process ensures that transmitted data remains confidential and tamper-proof during transmission (figure 6).

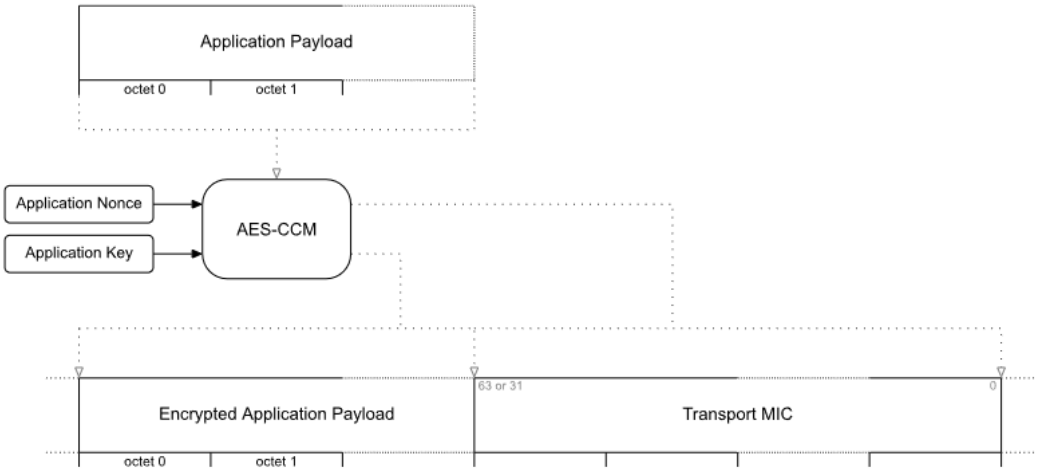

Figure 6. Encryption of Application Payload

Similarly, at the network layer, the destination address and TransportPDU are encrypted and authenticated using AES-CCM. All Network PDUs are encrypted using an Encryption Key derived from a network key and a Network Nonce. By encrypting both the destination address and payload, Bluetooth Mesh ensures that data remains protected against unauthorized access or modification throughout its transmission within the network (figure 7).

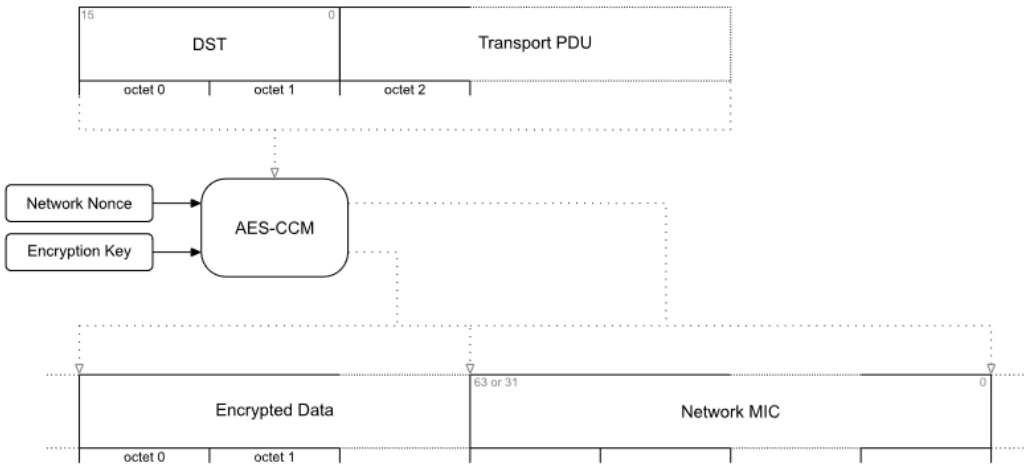

Figure 7. Encryption of Transport

To further enhance security, Bluetooth Mesh incorporates network layer obfuscation techniques. This process involves combining certain elements of the Network PDU with the IV Index and Privacy Key to prevent passive eavesdropping and node

identification. By encrypting specific portions of the PDU and introducing randomness, Bluetooth Mesh mitigates the risk of data interception and enhances overall network privacy and security (figure 8).

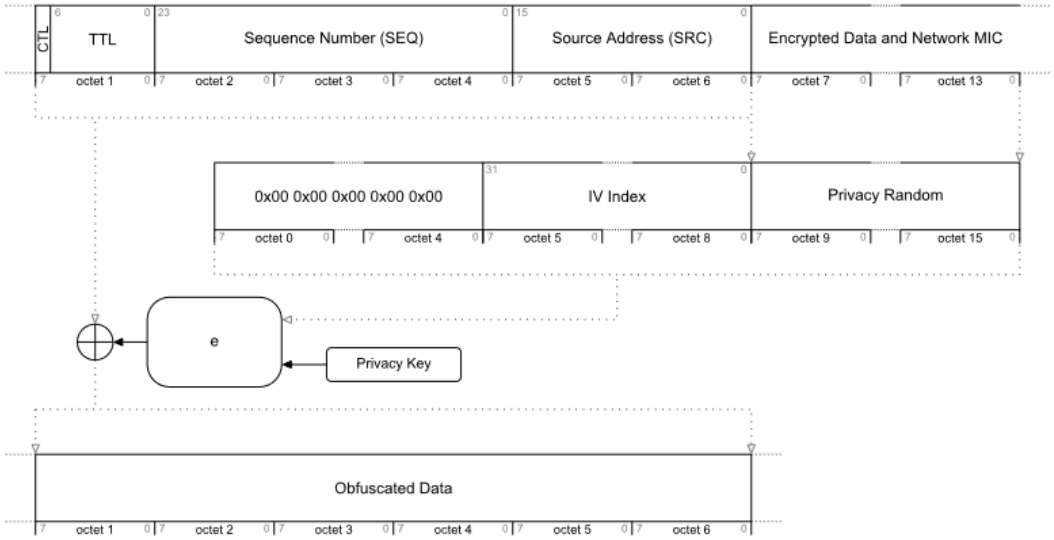

Figure 8. Layer Obfuscation Technique

2.3.7 Receiving and Transmitting Data

When receiving a Network PDU, the message undergoes initial processing as it transitions from the bearer to the network layer via a network interface (figure 9). This process involves the application of filtering rules defined by the input filter of the interface. If the message passes this filtering stage, it proceeds to the network layer for further processing.

Upon reception, each Network PDU can be enriched with additional metadata for potential future use. Subsequently, the node verifies the Network Identifier (NID) field value against known NIDs. If there is no match, the message is disregarded. Conversely, if the NID matches a known value, the node proceeds to authenticate the message against each corresponding network key. If authentication is successful, and other conditions such as the absence of the message in the Network Message Cache are met, the message advances for processing by the lower transport layer.

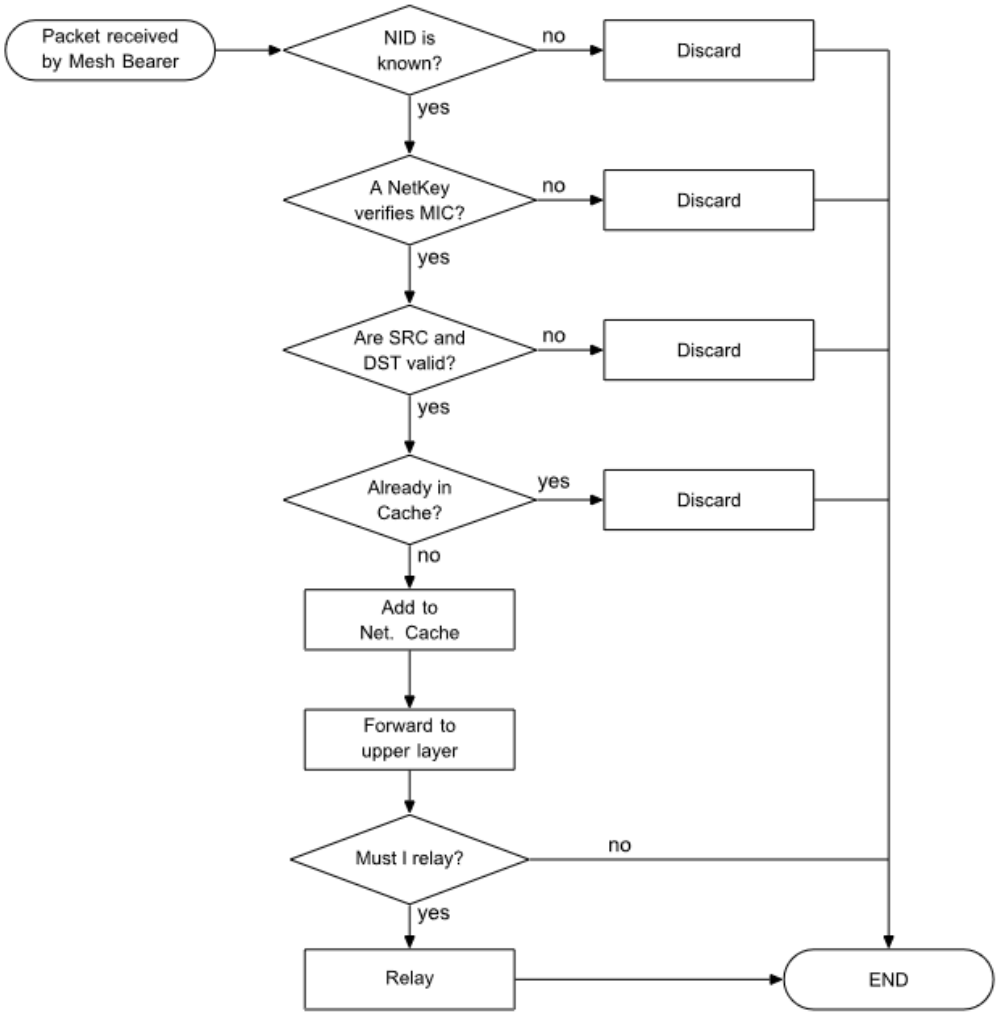

Figure 9. Filtering stage

In the event of message retransmission, the Invariant Index (IV Index) used remains consistent with the index when the message was initially received. Further, if certain conditions align, such as support for the Relay or Proxy features and a TTL value of 2 or greater, the message may undergo decrementation of the TTL field and subsequent retransmission to relevant network interfaces.

When transmitting a Network PDU, the process involves the configuration of various fields including the IVI field, NID field, CTL field, TTL field, SEQ field, SRC field, DST field, TransportPDU field, and NetMIC field. These fields collectively define the

message attributes and parameters for transmission within the mesh subnet. Upon configuration, the message is delivered to all network interfaces, subject to filtering rules defined by their respective output filters.

Furthermore, to optimize network performance and security, nodes maintain a Network Message Cache. This cache stores recently encountered Network PDUs to minimize redundant processing and relay operations. When a new PDU is received, it undergoes cache validation. If the PDU is not already cached, it proceeds for processing and storage in the cache. However, if the cache is full, the incoming PDU replaces the oldest cached entry. It is imperative for the cache to have a sufficient capacity aligned with network density to ensure efficient operation.

2.3.8 GATT

The GATT (Generic Attribute) bearer in Bluetooth mesh networking serves a vital role in enabling devices that may not support the advertising bearer to participate effectively in a mesh network. This bearer operates by utilizing the Proxy protocol to transmit and receive Proxy PDUs between two devices over a GATT (Generic Attribute Profile) connection. [3.]

In the Bluetooth mesh specification, two mesh bearers are defined: the advertising bearer and the GATT bearer. While the advertising bearer utilizes BLE advertising PDUs to transport mesh messages, the GATT bearer utilizes the Proxy protocol to facilitate communication between devices.

The GATT bearer defines two distinct roles: the GATT Bearer Client and the GATT Bearer Server. The GATT Bearer Server, implemented as a GATT Server, is responsible for hosting the Mesh Proxy Service, allowing it to handle incoming and outgoing Proxy PDUs. On the other hand, the GATT Bearer Client, functioning as a GATT Client, interacts with the Mesh Proxy Service to transmit and receive Proxy PDUs.

To enable communication over the GATT bearer, the GATT Bearer Client must perform primary service discovery to locate the Mesh Proxy Service. This can be achieved using standard GATT procedures such as the GATT Discover All Primary Services or the GATT Discover Primary Services by Service UUID sub-procedure. Once the Mesh Proxy Service is discovered, the GATT Bearer Client proceeds to discover its characteristics, including the Mesh Proxy Data In and Mesh Proxy Data Out characteristics.

To send a Proxy PDU, the GATT Bearer Client utilizes the Write Without Response sub-procedure to write the Proxy PDU to the Mesh Proxy Data In characteristic of the GATT Bearer Server. Conversely, to receive a Proxy PDU, the GATT Bearer Client configures the Mesh Proxy Data Out characteristic to enable notifications. Each notification received contains a single Proxy PDU, allowing seamless communication between devices over the GATT bearer within the Bluetooth mesh network.

2.4 Comparison to Zigbee

Zigbee is a wireless communication protocol widely used in IoT applications, offering low-power, low-data-rate transmissions over short distances. Unlike Bluetooth Mesh, Zigbee operates on the IEEE 802.15.4 standard, using the 2.4 GHz frequency band (and sometimes from 779MHz to 928 MHz, depending on region and purpose of device). While both protocols support mesh networking, Zigbee typically employs a star or tree topology, whereas Bluetooth Mesh utilizes a managed flooding approach. Zigbee networks are characterized by their low latency and power consumption, making them suitable for devices with limited energy resources. However, Bluetooth Mesh offers greater scalability and interoperability, with support for larger networks and standardized device profiles, making it well-suited for diverse IoT deployments. [16.]

	BLUETOOTH MESH	ZIGBEE

YEAR INTRODUCED	July 2017	June 2005
FREQUENCY RANGE (PRIMARY)	2.4Ghz	2.4Ghz
SPEED	Depends on version, medium is 270kpbs [9.] [11.]	40—250kbps
MAXIMUM NODES (THEORETICAL)	32000	64000
TRANSMIT POWER	10mW	100mW
ROUTING TYPE	Managed flooding	Full routing
RANGE (APPROXIMATELY)	0-100 meters	0-100 meters

Table 6. Comparison of Bluetooth Mesh and Zigbee

3 HARDWARE AND SOFTWARE COMPONENTS

3.1 Hardware

3.1.1 Provisioner

The provisioner node in the Bluetooth Mesh network serves as a pivotal entity responsible for initializing and configuring other nodes within the network. In this implementation, the Raspberry Pi 5 serves as the provisioner, leveraging its robust hardware capabilities to facilitate seamless provisioning operations.

The Raspberry Pi 5 (figure 10), developed by the Raspberry Pi Foundation, features a powerful SoC architecture, making it an ideal candidate for provisioning tasks in Bluetooth Mesh networks. At its core lies the Broadcom BCM2712 chipset, which integrates a quad-core ARM Cortex-A76 CPU clocked at up to 2.4 GHz. [17.]

One of the standout features of the Raspberry Pi 5 is its built-in wireless connectivity options, including Bluetooth 5. Equipped with onboard BLE Broadcom BCM43455 chip, it meets minimum requirements for a Bluetooth mesh provisioner to operate, as it will use GATT.

Figure 10. Raspberry Pi 5

3.1.2 Node

The Nordic Thingy 53 (figure 11), a versatile development platform engineered by Nordic Semiconductor, assumes the role of a node within the Bluetooth Mesh network, offering robust functionality and flexibility for diverse IoT applications. As a node, the Nordic Thingy 53 integrates seamlessly into the mesh network topology, contributing to the network's distributed intelligence and collaborative operation.

At the heart of the Nordic Thingy 53 lies Nordic Semiconductor's advanced SoC architecture, featuring the nRF5340. This chipset incorporates a 2 powerful ARM Cortex-M33 processors, operating at frequencies up to 128 MHz, providing ample computational resources for executing various node functions and mesh networking protocols. [18.]

A defining feature of the Nordic Thingy 53 is its integrated Bluetooth 5.4 Low Energy (BLE) radio, which serves as the primary wireless communication interface for interacting with neighboring nodes and the provisioner within the Bluetooth Mesh network.

The Nordic Thingy 53's BLE capabilities adhere to industry-standard Bluetooth specifications, ensuring interoperability with a wide range of BLE-enabled devices and Bluetooth Mesh-compliant infrastructure. This interoperability facilitates seamless integration of the Nordic Thingy 53 into existing Bluetooth ecosystems and enables versatile application scenarios, from smart lighting and environmental monitoring to asset tracking and industrial automation.

In addition to its wireless connectivity, the Nordic Thingy 53 offers a rich set of onboard peripherals, such as buttons and sensors. It also features a programmable led, that is going to be used for demonstration.

Figure 11. Nordic Thingy 53

3.2 Software

3.2.1 Operating System

Raspberry Pi operates on various operating systems. However, for this project, the Raspberry Pi OS was chosen due to its official maintenance and support by the Raspberry Pi Foundation, along with its comprehensive feature set. The Raspberry Pi OS is based on the Debian GNU/Linux distribution, renowned for its stability and reliability in embedded systems.

On the other hand, the Nordic Semiconductor Thingy 53 employs the Zephyr operating system (Zephyr OS), a lightweight, scalable, and open-source RTOS designed specifically for resource-constrained devices like IoT and embedded systems. Zephyr OS provides a flexible and modular framework optimized for low-power, connected devices, making it suitable for applications requiring efficient Bluetooth mesh networking capabilities.

3.2.2 Bluetooth Stack on GNU/Linux

In the realm of GNU/Linux, multiple Bluetooth stacks are available to manage Bluetooth communication protocols [14]. Among these, BlueZ emerges as a prominent choice, particularly for its inclusion as the default Bluetooth stack in the Raspberry Pi OS. BlueZ stands as a mature and robust Bluetooth stack, providing comprehensive support for core Bluetooth layers and protocols essential for seamless communication in Bluetooth-enabled environments.

BlueZ's significance in Bluetooth mesh networking lies in its provision of features and capabilities tailored to mesh communication paradigms. It offers robust support for mesh-specific protocols and functionalities since version v5.47, enabling the creation, configuration, and management of Bluetooth mesh networks with efficiency and reliability. [15.]

3.2.3 Meshctl Tool

The meshctl CLI tool, provided by BlueZ since version v5.47, serves as a crucial utility for managing Bluetooth mesh networks. Its functionality encompasses various aspects of mesh network provisioning, configuration, monitoring, and debugging, facilitating efficient network management and troubleshooting.

Upon introduction, meshctl grants access to a comprehensive set of functionalities tailored to the intricacies of Bluetooth mesh networking. These functionalities include node provisioning, network configuration, monitoring network topology, and diagnosing network issues.

```
d3sx@d3sx-pi ~> meshctl
Waiting to connect to bluetoothd...Reading prov_db.json and
 local_node.json from /home/d3sx/.config/meshctl directory
[meshctl]# version
Version 5.66
[meshctl]#
```

Figure 12. Meshctl tool showing its version

Upon initialization, meshctl undergoes a structured routing process aimed at acquiring up-to-date data from two primary sources: "prov_db.json" and "local_node.json". The former encompasses comprehensive details regarding all network nodes, keys, and provisioners, while the latter specifically delineates information pertaining to the local node, notably the Raspberry Pi in our context.

The efficacy of the tool hinges upon robust encryption mechanisms, notably AEAD-AES_CCM, which are indispensable for ensuring the confidentiality, integrity, and authenticity of transmitted data within the Bluetooth mesh network. AEAD (Authenticated Encryption with Associated Data) functions as a pivotal safeguard, preserving data security during transmission while furnishing authentication to validate the credibility of the data source. AES_CCM (Advanced Encryption Standard Counter with Cipher Block Chaining-Message Authentication Code) stands out as a symmetric encryption algorithm widely embraced in Bluetooth mesh networks owing to its commendable efficiency and efficacy in data protection. Notably, these encryption algorithms are not inherently integrated into the default Raspberry Pi OS kernel, necessitating recompilation to enable their functionality. [20.]

4 PREPARATION

4.1 Raspberry Pi 5

4.1.1 Installation of Raspberry Pi OS

The installation of the Raspberry Pi OS on the Raspberry Pi 5 involved the utilization of the "Raspberry Pi Imager" software. This tool facilitated the flashing of the Raspberry Pi OS 64-bit version, tailored for compatibility with the Raspberry Pi 5 hardware. Additionally, a 64-gigabyte SD card from Samsung was employed as the storage medium for the operating system. The process encompassed the selection of the appropriate OS version within the Raspberry Pi Imager interface, followed by the flashing of the image onto the SD card.

4.1.2 Recompilation of Kernel

The recompilation of the kernel for the Raspberry Pi involved a detailed process to ensure compatibility and functionality within the Bluetooth mesh implementation of BlueZ [19]. Following the official Raspberry Pi documentation available [20] the kernel sources were obtained from the Raspberry Pi GitHub repository using Git cloning. Options related to cryptographic APIs and encryption algorithms were enabled during the configuration process to meet the security requirements of the Bluetooth mesh implementation. These options are – CCM, CMAC, AES, Hash Algorithms, Symmetric key cipher algorithms and AEAD cipher algorithm.

4.1.3 Installation of BlueZ and Meshctl

BlueZ comes pre-installed on Raspberry Pi OS. Meshctl can be obtained either through the APT package manager or by compiling the source code for BlueZ with some flags that enable mesh support [19], which includes meshctl as one of its utilities.

4.2 Nordic Thingy 53

4.2.1 Installation of Programmer

As a programmer for Nordic Thingy 53, “nRF Programmer” was used. This application, developed by Nordic Semiconductor, is available for installation on Android smartphones through the Google Play Store. Once installed, the nRF Programmer app enables one to program and configure Nordic Semiconductor devices, including the Thingy 53, directly from their Android smartphones.

4.2.2 Choosing the Correct Application

In the process of selecting the appropriate applications for the Nordic Thingy 53, two specific applications were chosen and flashed using the nRF Programmer. The first selected application is "Bluetooth Mesh: light," which is tailored for lighting functionalities within the Bluetooth mesh network. This application enables control over lighting elements, providing features such as on/off functionality.

Additionally, the second application chosen is "Bluetooth mesh: light switch," which is designed to serve as a control interface for the lighting system. With this application, it is possible to interact with the lighting network, toggling lights on or off.

5 PRACTICAL USAGE

5.1 Example Scenario

5.1.1 Set-up Switch and Light

In this scenario, we delve into the practical application of Bluetooth mesh networking. Specifically, we explore the process of configuring a switch and a light within the Bluetooth mesh ecosystem. From a Bluetooth mesh perspective, this entails provisioning the devices onto the mesh network, configuring their roles and functionalities, and establishing communication protocols for seamless operation within the network architecture as shown in figure 13.

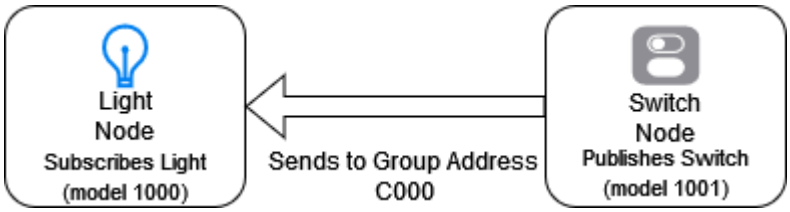

Figure 13. Switch and Light example

In this case, the Switch node will publish its 1001 Generic OnOff Client (Switch model) status to group address c000 and Light node will subscribe its 1000 Generic OnOff Server (LED model) to the same group address. [13.] [22.]

5.2 Implementation in Meshctl

5.2.1 Overview

Meshctl CLI provides a comprehensive set of commands for configuring and managing Bluetooth mesh networks. The structure of meshctl is organized into different menus and submenus, each offering specific functionalities for network setup, device management, and configuration.

Table 7. Meshctl Main Menu

List	Lists available controllers
show [ctrl]	Displays controller information
select <ctrl>	Selects the default controller
security [0(low)/1(medium)/2(high)]	Displays or changes provision security level
info [dev]	Provides device information
connect [net_idx] [dst]	Connects to a mesh network or node on the network (net_idx is 0 by default)
discover-unprovisioned <on/off>	Looks for devices to provision
provision <uuid>	Initiates provisioning
power <on/off>	Sets controller power
disconnect [dev]	Disconnects a device
mesh-info	Displays mesh network information (prov_db.json)
local-info	Shows local mesh node information (local_node.json)
menu <name>	Selects a submenu (config or onoff)
version	Displays version of meshctl
quit and exit	Exits the program

help	Provides help about the program
export	Prints environment variables

Table 8. Meshctl Config Menu

target <unicast>	Sets the target node to configure
composition-get [page_num]	Retrieves composition data
netkey-add <net_idx>	Adds a network key
netkey-del <net_idx>	Deletes a network key
appkey-add <app_idx>	Adds an application key
appkey-del <app_idx>	Deletes an application key
bind <ele_idx> <app_idx> <mod_id> [cid]	Binds an app key to a model
mod-appidx-get <ele_addr> <model id>	Retrieves model app_idx
ttl-set <ttl>	Sets default TTL
ttl-get	Retrieves default TTL
pub-set <ele_addr> <pub_addr> <app_idx> <per (step|res)> <re-xmt (cnt|per)> <mod id> [cid]	Sets publication
pub-get <ele_addr> <model>	Retrieves publication
proxy-set <proxy>	Sets proxy state

proxy-get	Retrieves proxy state
ident-set <net_idx> <state>	Sets node identity state
ident-get <net_idx>	Retrieves node identity state
beacon-set <state>	Sets node beacon state
beacon-get	Retrieves node beacon state
relay-set <relay> <rexmt count> <rexmt steps>	Sets relay
relay-get	Retrieves relay
hb-pub-set <pub_addr> <count> <period> <ttl> <features> <net_idx>	Sets heartbeat publish
hb-pub-get	Retrieves heartbeat publish
hb-sub-set <src_addr> <dst_addr> <period>	Sets heartbeat subscribe
hb-sub-get	Retrieves heartbeat subscribe
sub-add <ele_addr> <sub_addr> <mod id> [cid]	Adds subscription
sub-get <ele_addr> <model id>	Retrieves subscription
node-reset	Resets a node and removes it from the network
back	Returns to the main menu

Table 9. Meshctl Menu onoff

target <unicast>	Sets the target node to configure
get	Retrieves the ON/OFF status of the specified node
onoff <0/1>	Sends a "SET ON/OFF" command to the target node. The parameter (0 or 1) determines whether the command sets the node to ON or OFF
back	Returns to the main menu

5.2.2 Implementation

To implement the set-up switch and light example from example scenario , firstly the discovery of unprovisioned devices is initiated, facilitating the identification of available nodes awaiting provisioning. From this it is possible to take the unique identifier (UUID) and provision a device by using it as shown in figure 14.

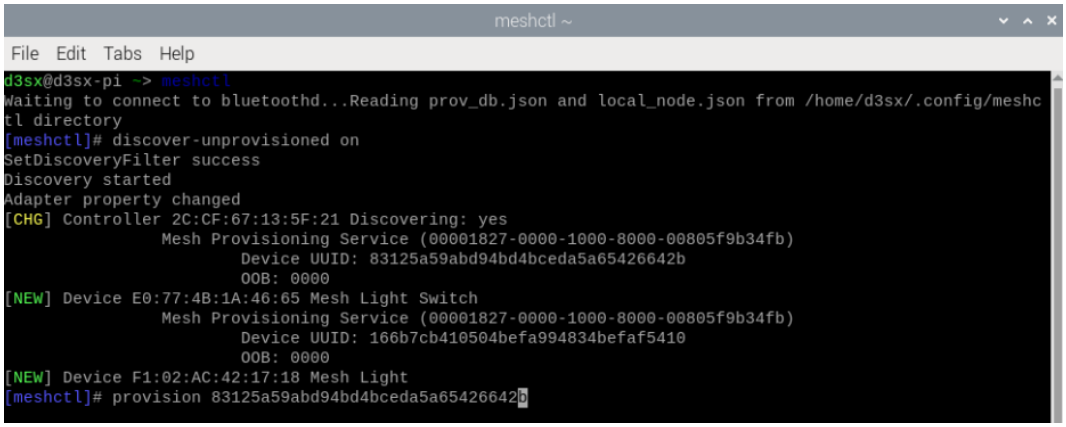

Figure 14. Discovering and provisioning a device by using meshctl

Right after the command provision is initiated, the provisioning process begins. When it is finished, we can see the Unicast Address of the first element in node as shown in figure 15.

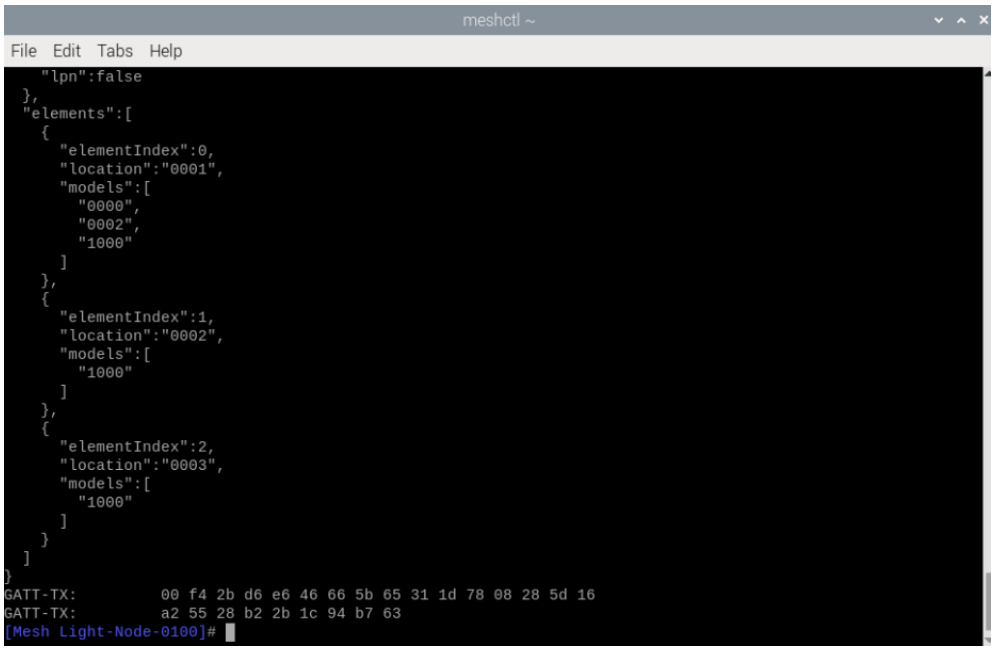

Figure 15. Successfully provisioned device in meshctl

After provisioning both nodes we can start configuring them. For the light node, we add Application key with index 1 to it, bind node with index 0 (Unicast Address 0100) and model 1000 (Generic OnOff Server) to Application key with index 1 and subscribe this node's model 1000 to c000 Group Address (Figure 16).

On the Light switch side, we also configure it to use the same Application key, but now we bind it to model 1001 (Generic OnOff Client). Also, instead of subscribing, we publish its model 1001 to group address c000 without retransmission and publication period (figure 17).

```
meshctl ~
File  Edit  Tabs  Help
export                                          Print environment variables
[Mesh Light-Net-0000]# target 0100
Configuring node 0100
[config: Target = 0100]# appkey-add 1
GATT-TX:          00 f4 0f 0f e3 51 1a 1a a4 41 69 f6 42 a6 52 d6
GATT-TX:          1b 95 88 d5 a2 ab 8c a4 15 3a e9 ac 12 b5
GATT-TX:          00 f4 96 7a 04 38 1e 16 7e e2 f9 d4 a8 d1 ad 1a
GATT-TX:          5f b4 ae 02 fd 74 5c b4 87 63 74 33 4d b5
GATT-RX:          00 f4 d9 e0 31 65 ee 28 2d 5c a9 41 dc ec aa 7d
GATT-RX:          65 06 a3 ab 49 eb b0 be cb
GATT-RX:          00 f4 72 fd 6f 64 bc 3e 22 a8 00 ec 6e 63 3a 81
GATT-RX:          56 02 5a d4 6d 40 63 9a a2
Node 0100 AppKey status Success
NetKey  000
AppKey  001
[config: Target = 0100]# bind 0 1 1000
GATT-TX:          00 f4 9e 90 dd 45 71 1f ba 40 33 a4 55 38 0f 4f
GATT-TX:          a0 84 74 ba fe 7f de 2a 1f 35 1a
GATT-RX:          00 f4 e4 5d 82 70 ad a2 45 3c 01 ff 33 28 61 18
GATT-RX:          3d c2 c6 a7 c2 4c 9b 24 fe 20 82 d8
Node 0100 Model App status Success
Element Addr    0100
Model Id        1000
AppIdx          001
 [config: Target = 0100]#sub-add 0100 c000 1000
[config: Target = 0100]# sub-add 0100 c000 1000
GATT-TX:          00 f4 11 65 39 12 d5 10 4b 04 20 37 6a a7 ad 49
GATT-TX:          b2 c1 a6 36 3d 2f 2b 03 0a f5 29
GATT-RX:          00 f4 ed 73 24 95 87 b0 03 f4 0c d4 76 0f 25 91
GATT-RX:          cf 98 85 e2 84 0b 54 82 24 fa d6 b5

Node 0100 Subscription status Success
Element Addr    0100
Model Id        1000
Subscr Addr     c000
[config: Target = 0100]#
```

Figure 16. Set up for light node

```
meshctl ~
File  Edit  Tabs  Help
[Mesh Light Switch-Node-0103]# target 0103
Configuring node 0103
[DEL] Device F1:02:AC:42:17:18 Mesh Light
[config: Target = 0103]# appkey-add 1
GATT-TX:          00 f4 ff 3d 5f a3 aa 0c d5 3a b2 0f 7e 76 ef 5b
GATT-TX:          89 71 41 af 0d 25 74 10 d9 60 80 53 82 8e
GATT-TX:          00 f4 18 1c f9 d7 71 d4 02 f4 d5 54 6f ff c8 c2
GATT-TX:          6e 80 1c 0f 56 1d 07 4f de a8 bb 80 df 92
GATT-RX:          00 f4 29 2e 08 19 a2 0c d7 80 d6 40 c7 98 30 9f
GATT-RX:          93 6a 36 8d 53 16 07 07 62
GATT-RX:          00 f4 52 ae 84 61 20 a1 dd cd f5 cf 47 88 f9 2d
GATT-RX:          10 dc a7 b0 e8 a9 d5 74 d8
Node 0103 AppKey status Success
NetKey  000
AppKey  001
[config: Target = 0103]# bind 0 1 1001
GATT-TX:          00 f4 81 ee ee 7b 77 25 cb 32 25 90 b9 74 cc da
GATT-TX:          e9 8b 7a dd 51 9d ca cf a9 1f 29
GATT-RX:          00 f4 88 b8 b9 84 fc aa ad d5 a0 a4 06 fb 8e 51
GATT-RX:          8c ac 0d 9b 3e bc a2 9b 50 f8 cd ea
Node 0103 Model App status Success
Element Addr    0103
Model Id        1001
AppIdx          001
 [config: Target = 0103]# pub-set 0103 c000 1 0 0 1001
GATT-TX:          00 f4 57 41 2d b9 92 b5 2a 8f 2c 5e 5e ab 7b a8
GATT-TX:          ab 48 5a 9c d5 8a 38 62 e4 ee c5 87 81 4a
GATT-TX:          00 f4 28 d4 0c a8 2b df c6 f2 16 93 66 86 1f 3e
GATT-TX:          06 d5 49 21 1f d8
GATT-RX:          00 f4 a8 f7 94 49 c4 99 97 0e e4 5a 97 4c 5d 3f
GATT-RX:          a1 11 5e 10 18 9f 51 f7 d6
GATT-RX:          00 f4 18 84 f6 b2 56 cd 90 8a f7 28 60 13 dd f6
GATT-RX:          3f 13 21 53 37 72 9c d1 34 dc f5 9c a8 eb
GATT-RX:          00 f4 0e 5c 27 d3 3f 1f 34 59 bc f1 ea 15 5e 19
GATT-RX:          b3 90 91 9d 0e 55 a4 95

Node 0103 Publication status Success
Element Addr    0103
Model Id        1001
Pub Addr        c000
Period          0 ms
Rexmit count    0
Rexmit steps    0
GATT-TX:          00 f4 ab 33 05 7a 1e 01 ca 6e d0 6a 0d a7 18 94
GATT-TX:          39 81 11 f8 3b 3a f7 59 97
Notify closed
Write closed
Services resolved no
Characteristic property changed /org/bluez/hci0/dev_E0_77_4B_1A_46_65/service0014/char0017
Characteristic property changed /org/bluez/hci0/dev_E0_77_4B_1A_46_65/service0014/char0015
[meshctl]#
```

Figure 17. Set up for Switch node

5.3 Implementation In Web Application

5.3.1 Overview

The web application serves as a complementary tool tailored explicitly for this thesis project. Its main goal was to automate processes, and to introduce additional functionalities not readily available in the Meshctl CLI.

Developed using Next.js for the frontend and Python for the backend, the web application provides an intuitive and user-friendly interface for managing the Bluetooth mesh network. It features Home Page, Nodes Page, Keys Page and Status Page

The application main page features provisioning functionality. When the button is pressed, it starts a scan process and displays the nodes, then once the provision button is pressed it opens output from provisioning process, which goes straight from meshctl (Figure 18).

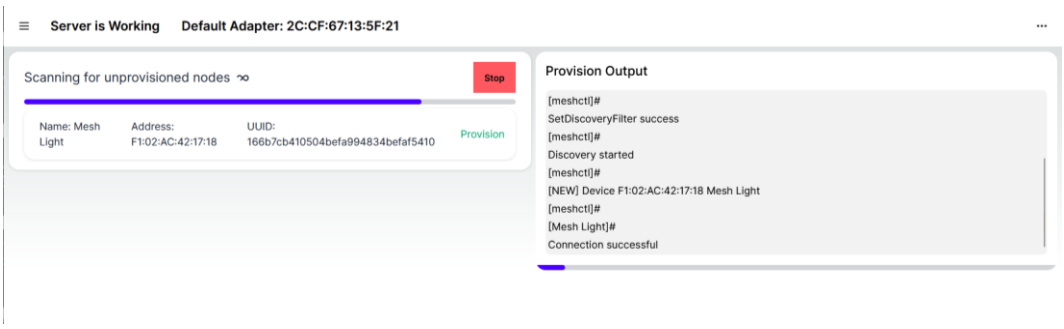

Figure 18. Discovering and provisioning node in Web Application

The Nodes page shows all nodes that once were provisioned. It displays Company ID, Product ID, Version ID, features of node (such as relay), Nonces, Network keys (that are used by the node and Available elements). Also, it features configuration of bind, subscribe and publish.

The page gets the company name from Company ID and company identifiers list that is publicly available [22]. The same thing applies to available models [21].

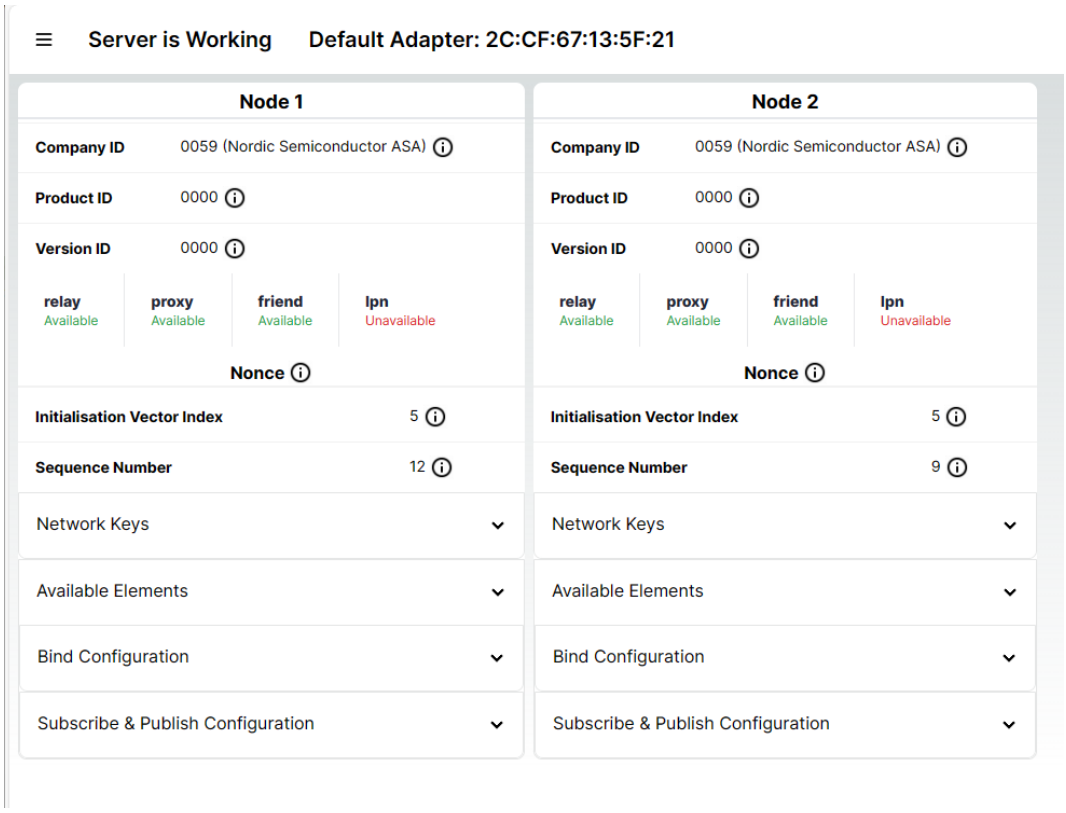

Figure 19. Nodes page in Web Application

The Keys page obtains the data about all Application and Network keys that are used in network. The Application keys availability list shows their index, bound to the network key index and value, meanwhile the Network keys list shows their index, their refresh status and value. The Application keys assigned list shows node elements to which the keys are assigned, the types of operation, the model id to which they assigned the index/address to which they assigned, while the network keys show nodes, nodes device key values and indexes of keys.

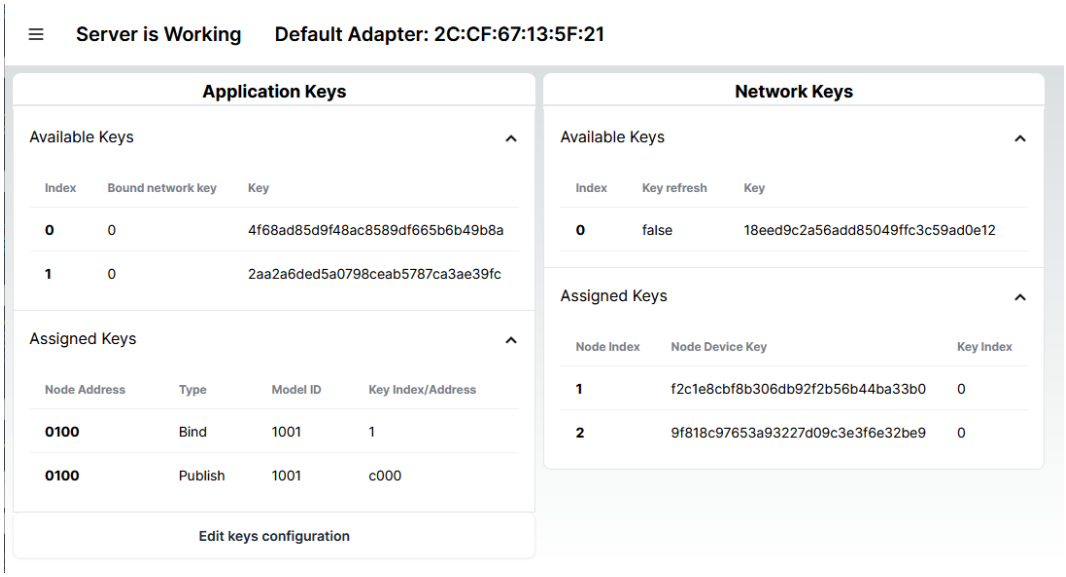

Figure 20. Keys page in Web Application

The Status page shows the current version of application and meshctl in-use. Also, it allows one to change the default adapter in use and to reset the meshctl config files.

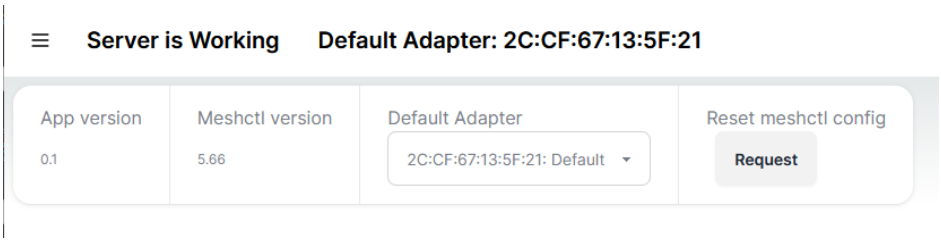

Figure 21. Status Page in Web Application

5.3.2 Implementation

To enact the set-up switch and light example delineated in the example scenario, we initialize the provisioning procedure. This necessitates navigating to the provisioning page and activating the scan by engaging the designated button. Subsequently, upon detection of any devices, they are promptly listed, signaling the

commencement of the provisioning process. Upon selecting the "provision" button, the program orchestrates the provisioning sequence, furnishing real-time feedback within the "Provision Output" section.

Following the successful provisioning of both devices, we navigate to the "Node" page, which displays comprehensive information about all nodes. Here, our first step involves effortlessly binding models to any Application keys, mirroring the actions performed in meshctl. Subsequently, within the "Subscribe & Publish Configuration" section, we configure the light node to subscribe to its model 1000 (Generic OnOff Server) at group address c000. Conversely, the switch node is configured to publish its element 1001 (Generic OnOff Client) to the same group address c000.

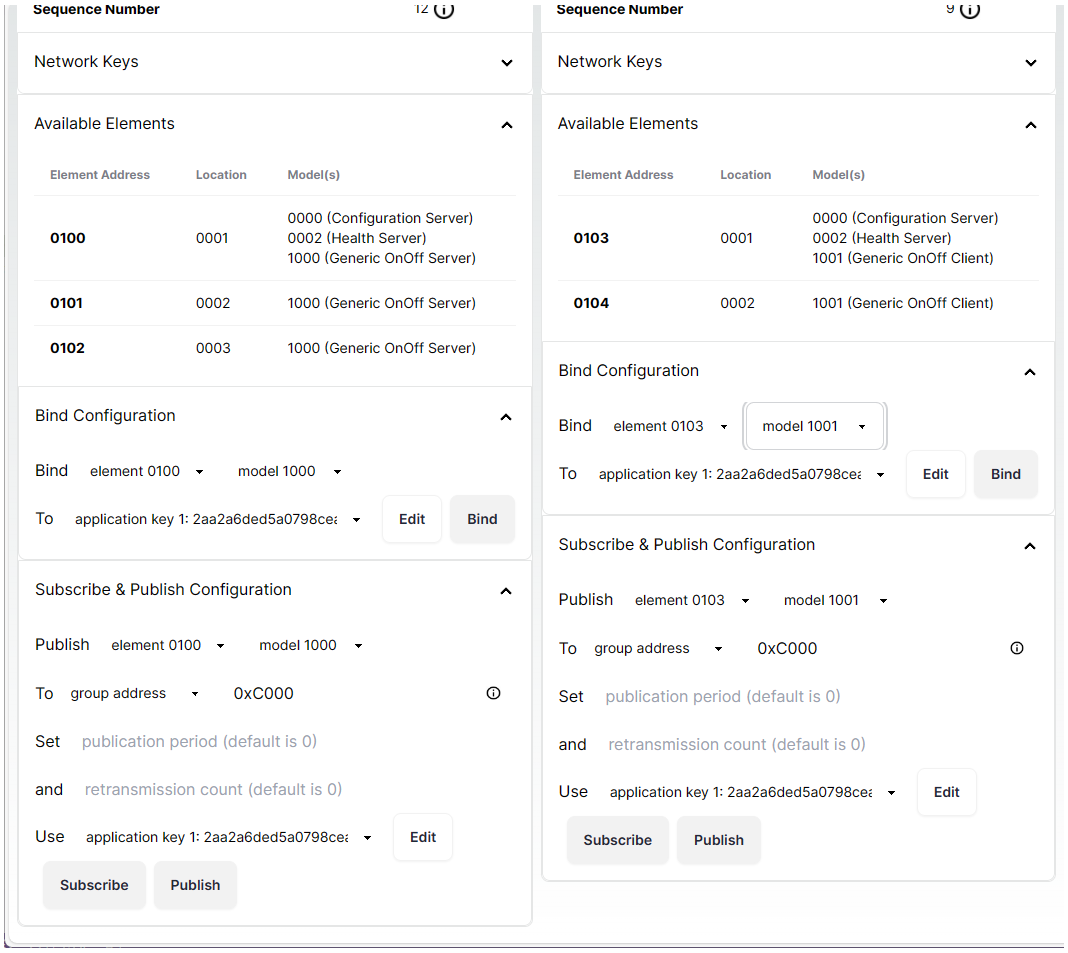

Figure 22. Set up Light and Switch nodes in Web Application

6 CONCLUSIONS

In conclusion, this thesis has presented a comprehensive exploration of Bluetooth mesh networking, focusing on its architecture, data transmission mechanisms, and practical implementation using Raspberry Pi and Nordic Thingy 53 devices through detailed analysis and practical examples. The provisioning process, configuration of network keys, and establishment of communication protocols within the Bluetooth mesh network were elucidated throughout the thesis.

Moreover, the role of software tools such as Meshctl have been highlighted and the development of a web application to streamline the management and operation of Bluetooth mesh networks. This research contributes to the advancement of Bluetooth mesh technology by providing insights into its practical implementation and addressing key challenges in network configuration and management.

Further research and development in this area can enhance the scalability, security, and interoperability of Bluetooth mesh networks, paving the way for their widespread adoption in various IoT applications. With continued innovation and collaboration, Bluetooth mesh networking holds the potential to revolutionize the way we connect and interact with smart devices in our everyday lives.

REFERENCES

1. Gomez C, Oller J, Paradells J. Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology. 2012 Aug 29 [cited 2024 May 2]; Available from: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3478807/
2. Woolley M, Kulkarni O, Winiarczyk P. Bluetooth® Mesh Remote Provisioning [Internet]. Bluetooth®. 2023 [cited 2024 May 2]. Available from: https://www.bluetooth.com/mesh-remote-provisioning/
3. Adafruit. GATT | Introduction to Bluetooth Low Energy [Internet]. adafruit.com. 2014 [cited 2024 May 2]. Available from: https://learn.adafruit.com/introduction-to-bluetooth-low-energy/gatt
4. Bluetooth. Specifications | Assigned Numbers [Internet]. 2024 Apr [cited 2024 May 2]. Available from: hhttps://www.bluetooth.com/specifications/assigned-numbers/
5. Bluetooth. Bluetooth Mesh Glossary of Terms [Internet]. www.bluetooth.com. [cited 2024 May 2]. Available from: https://www.bluetooth.com/learn-about-bluetooth/feature-enhancements/mesh/mesh-glossary/
6. Ren K, Woolley M. The Fundamental Concepts of Bluetooth Mesh Networking Part 1 [Internet]. Bluetooth. 2017 [cited 2024 May 2]. Available from: https://www.bluetooth.com/blog/the-fundamental-concepts-of-bluetooth-mesh-networking-part-1/
7. Ren K. Provisioning a Bluetooth Mesh Network Part 1 [Internet]. Bluetooth. 2017 [cited 2024 May 2]. Available from: https://www.bluetooth.com/blog/provisioning-a-bluetooth-mesh-network-part-1/
8. Ren K. Provisioning a Bluetooth Mesh Network Part 2 [Internet]. Bluetooth. 2017 [cited 2024 May 2]. Available from: https://www.bluetooth.com/blog/provisioning-a-bluetooth-mesh-network-part-2/
9. Perez-Diaz-De-Cerio D, Hernandez-Solana A, Garcia-Lozano M, Bardaji AV, Valenzuela JL. Speeding Up Bluetooth Mesh. IEEE Access [Internet]. 2021

[cited 2024 May 2];9:93267–84. Available from: https://ieeexplore.ieee.org/document/9466503

10. Wang M, Li Y, Lv J, Gao Y, Qiao C, Liu B, et al. ACE: A Routing Algorithm Based on Autonomous Channel Scheduling for Bluetooth Mesh Network. Electronics [Internet]. 2021 Dec 30 [cited 2024 May 2];11(1):113. Available from: https://www.mdpi.com/2079-9292/11/1/113
11. Silicon Laboratories. AN1424: Bluetooth ® Mesh 1.1 Network Performance. 2023 [cited 2024 May 2]; Available from: https://www.silabs.com/documents/public/application-notes/an1424-bluetooth-mesh-11-network-performance.pdf
12. Bluetooth. Mesh Profile 1.0.1 [Internet]. 2019 [cited 2024 May 2]. Available from: https://www.bluetooth.com/specifications/specs/mesh-profile-1-0-1/
13. Bluetooth. Mesh Model 1.1 [Internet]. 2023 [cited 2024 May 2]. Available from: https://www.bluetooth.com/specifications/specs/mesh-model-1-1/
14. Wikipedia. Bluetooth stack [Internet]. wikipedia.org. 2020 [cited 2024 May 2]. Available from: https://en.wikipedia.org/wiki/Bluetooth_stack
15. BlueZ. Release of BlueZ 5.47 [Internet]. www.bluez.org. 2017 [cited 2024 May 2]. Available from: https://www.bluez.org/release-of-bluez-5-47/
16. ZigBee. ZigBee Specification. 2015 Aug 5 [cited 2024 May 2]; Available from: https://zigbeealliance.org/wp-content/uploads/2019/11/docs-05-3474-21-0csg-zigbee-specification.pdf
17. Raspberry Pi Ltd. Raspberry Pi 5. 2023 Oct; Available from: https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf
18. Nordic Semiconductors. Nordic Thingy:53 Hardware. 2023 Nov 24 [cited 2024 May 2]; Available from: https://infocenter.nordicsemi.com/pdf/Thingy53_UG.pdf
19. Bluetooth. Bluetooth Mesh Daemon [Internet]. github.com. 2019 [cited 2024 May 2]. Available from: https://github.com/hadess/bluez/blob/master/mesh/README

20. Raspberry Pi Ltd. Raspberry Pi Documentation - The Linux kernel [Internet]. www.raspberrypi.com. 2024 [cited 2024 May 2]. Available from: https://www.raspberrypi.com/documentation/computers/linux_kernel.html
21. Bluetooth. mmdl_model_uuids [Internet]. bitbucket.org. 2024 [cited 2024 May 2]. Available from: https://bitbucket.org/bluetooth-SIG/public/src/main/assigned_numbers/mesh/mmdl_model_uuids.yaml
22. Bluetooth. company_identifiers [Internet]. bitbucket.org. 2024 [cited 2024 May 2]. Available from: https://bitbucket.org/bluetooth-SIG/public/src/main/assigned_numbers/company_identifiers/company_identifiers.yaml